



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom



Simulation d'un système de transmission

Transmission élémentaire *back-to-back*

Julien Mallet & Sylvain Guérin
SIT213 – TP1 – 3 septembre 2026

Commande unique (extrait)

SYNOPSIS

simulateur [...]

OPTIONS :

Par défaut le simulateur doit utiliser une chaîne de transmission logique, avec un message aléatoire de longueur 100, sans utilisation de sondes et sans utilisation de codeur.

-mess m

précise le message ou la longueur du message à émettre :

Si m est une suite de 0 et de 1 de longueur au moins égale à 7, m est le message à émettre. Si m comporte au plus 6 chiffres décimaux et correspond à la représentation en base 10 d'un entier, cet entier est la longueur du message que le simulateur doit générer et transmettre. Par défaut le simulateur doit générer et transmettre un message de longueur 100.

-s

indique l'utilisation des sondes.

Par défaut le simulateur n'utilise pas de sondes

-seed v

précise l'utilisation d'une semence pour l'initialisation des générateurs aléatoires du simulateur. v doit être une valeur entière. L'utilisation d'une semence permet de rejouer à l'identique une simulation (à la fois pour le message émis et le bruitage s'il est activé). Par défaut le simulateur n'utilise pas de semence pour initialiser ses générateurs aléatoires.

Code Simulateur (extrait)

```
public class Simulateur {
    public Simulateur(String [] args) throws ArgumentsException {
        analyseArguments(args);
        // A compléter
    }
    public static void main(String [] args) {

        Simulateur simulateur = null;

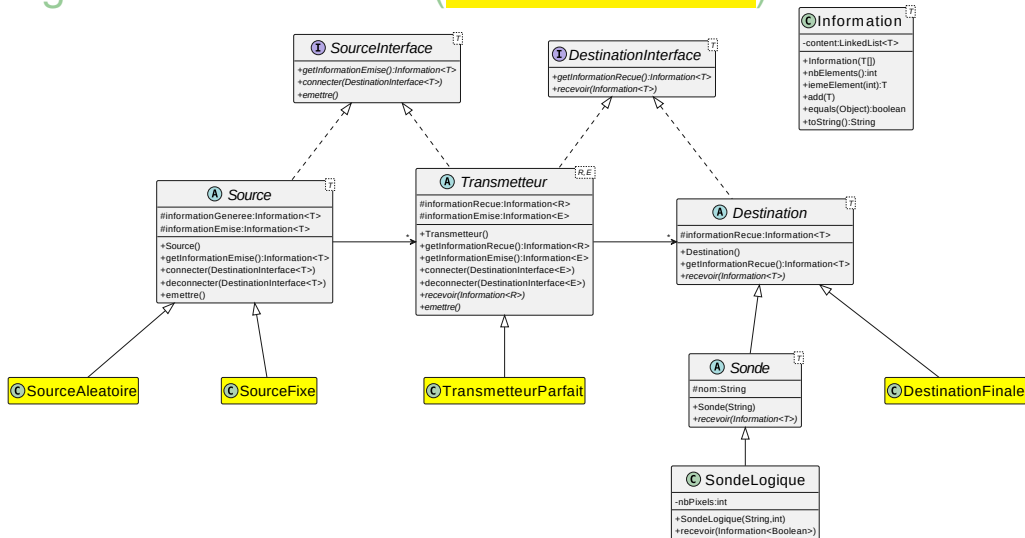
        try {
            simulateur = new Simulateur(args);
        }
        catch (Exception e) {
        }
        try {
            simulateur.execute();
            float tauxErreurBinaire = simulateur.calculTauxErreurBinaire();
            System.out.println(s + " => TEB : " + tauxErreurBinaire);
        }
        catch (Exception e) {
        }
    }
}
```



Simulateur

+Simulateur(String[])
-analyseArguments(String[])
+execute()
+calculTauxErreurBinaire():float
+main(String[])

Diagramme de classes (classes à créer)



Exemple d'exécution

- ▶ Instancier les composants de la chaîne de communication
 - ▶ SourceFixe
 - ▶ TransmetteurParfait
 - ▶ DestinationFinale
- ▶ Connecter les composants entre eux (méthode `connecter(DestinationInterface)`)
- ▶ Invoquer méthode `emettre` de SourceFixe
- ▶ Récupérer `informationEmise` et `informationRecue` et calcul du TEB (`calculerTauxErreurBinaire`)

Format du livrable logiciel attendu

À chaque étape X, le **logiciel** est livré sous Moodle sous forme d'une archive de nom : Nom1-Nom2.SIT213.EtapeX.tar.gz

L'archive contient, **directement à sa racine** :

- ▶ des dossiers src, bin, docs (bin et docs doivent être **vides**) ;
- ▶ des scripts compile, genDoc, cleanAll
- ▶ un script de lancement d'une simulation nommé simulateur et conforme aux exigences indiquées spécifiquement dans le document commande unique ;
- ▶ un script d'autotests nommé runTests ;
- ▶ un fichier readme explicatif global du contenu du dossier et de l'utilisation du logiciel.

Import source sous eclipse

- ▶ Création *Java Project* (**décocher *Create module-info.java file***)
- ▶ Sélection `src`
- ▶ *File/Import/Archive File/From archive file*
- ▶ *Browse* : `pour-eleves.tar.gz`, *Finish*

Diagramme de séquence

